Deployment Steps for Asynchronous Logger onTalos Kubernetes Cluster

1. Environment Setup

This has already been covered on the elearning system. You can access each step via this link [Environment setup]. But here is a highlighted list of initial steps.

1.1 Install Required Tools

1. VirtualBox 7.2.2
2. Talos Linux ISO
3. MobaXterm (SSH client to copy and paste outside of void)
4. Docker Desktop
5. Python 3.12 (for development only)
6. The **Talos bundle** :
   - `talosctl`
   - `kubectl`
   - `kubeconfig`
   - dashboard YAML/token files inside `/root/_out` on the Void VM.

2. Verify the Talos cluster is healthy

You'll do this from MobaXterm → SSH into the Void VM (e.g. `192.168.0.220`).

1. Open MobaXterm and connect to the Void VM (login as `root` or `void`) Go to the Talos out folder:

   ```
   cd /root/_out
   ```
2. Check that `kubectl` and `talosctl` are there: `ls`
3. Export KUBECONFIG so `kubectl` talks to your Talos cluster:

   ```
   export KUBECONFIG=/root/_out/kubeconfig
   ```

4. Verify the Kubernetes nodes:

```
./kubectl get nodes
```

You should see something like:

```
# export KUBECONFIG=/root/_out/kubeconfig
# ./kubectl get nodes
NAME            STATUS   ROLES           AGE     VERSION
talos-pmz-uib   Ready    control-plane   8d      v1.34.1
talos-qrt-zlx   Ready    <none>          8d      v1.34.1
talos-u4y-clt   Ready    <none>          4d20h   v1.34.1
# ./kubectl -n async-logger get pods
NAME                              READY   STATUS    RESTARTS      AGE
async-logger-7c44865b57-s4fnz     1/1     Running   3 (16m ago)   6d19h
# ./kubectl get pods -o wide -n async-logger
NAME                              READY   STATUS    RESTARTS      AGE     IP            NODE            NOMINATED NODE   READINESS GATES
async-logger-7c44865b57-s4fnz     1/1     Running   3 (17m ago)   6d19h   10.244.1.33   talos-qrt-zlx   <none>           <none>
#
```

If this works, your control-plane and worker machines are up and healthy.

3. Build & Push Docker Image

This should be done on **your Windows host** (not inside Talos).

3.1 Prepare the app folder

On Windows:

1. Create a folder, e.g. `C:\async-logger-app`.

2. Inside it, create three files. Click the links to go directly to the syntaxes:
   - app.py - a python file inside the `C:\async-logger-app` folder
   - Dockerfile - a .txt file that can be done through notepad. When saving, remove the .txt extension behind the title so it becomes a dockerfile instead of a text file.
   - requirements.txt - a regular text file. Save with .txt extension

3.2 Build and test the image locally

Open **PowerShell**, go to the folder:

```
cd C:\async-logger-app
```

```
docker build -t async-logger:v1 .
```

- If the build succeeds, run it once locally to verify: `docker run -p 8080:8080 async-logger:v1`

Open a new PowerShell window and call it:

```
Invoke-WebRequest `
  -Method POST `
  -Uri http://localhost:8080/log `
  -Headers @{ "Content-Type" = "application/json" } `
  -Body '{ "message": "hello from docker" }'
```

If you get `202 ACCEPTED` and see logs in the container, the image works.

```
PS C:\Users\SAI> Invoke-WebRequest `
>>   -Method POST `
>>   -Uri http://192.168.0.220:8080/log `
>>   -Headers @{ "Content-Type" = "application/json" } `
>>   -Body '{ "message": "hello from k8s" }'
>>


StatusCode        : 202
StatusDescription : ACCEPTED
Content           : {"message":"hello from k8s","status":"queued"}

RawContent        : HTTP/1.1 202 ACCEPTED
                    Connection: close
                    Content-Length: 47
                    Content-Type: application/json
                    Date: Mon, 10 Nov 2025 01:39:56 GMT
                    Server: Werkzeug/3.1.3 Python/3.12.12

                    {"message":"hello from k8s",...
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 47], [Content-Type, application/json], [Date, Mon, 10 Nov
                    2025 01:39:56 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 47
```

Stop the container (Ctrl + C in the `docker run` window).

3.3 Tag and push to Docker Hub

1. Log in to Docker Hub in **PowerShell**:
   `docker login`
2. Tag the image with your Docker Hub username:
   `docker tag async-logger:v1 YOUR_DOCKER_USERNAME/async-logger:v1`
3. Push it:
   `docker push YOUR_DOCKER_USERNAME/async-logger:v1`
4. Replace **`YOUR_DOCKER_USERNAME`** with your real Docker Hub name.
   When it finishes, you should see the image listed in Docker Desktop under **Images** → **My Hub**, and on hub.docker.com or your docker desktop app.

---

4. Kubernetes Deployment

4.1 Create Kubernetes manifests

Upload to Talos control node using MobaXterm drag & drop.

Files:

- namespace.yaml
- deployment.yaml - do not forget to replace `YOUR_DOCKER_USERNAME` with your real docker name
- service.yaml

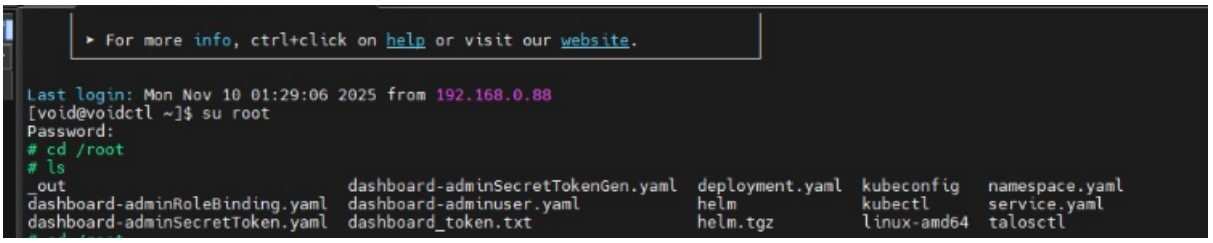
  Save all three files on your Windows machine.

---

5. Copy YAML files to Talos control plane

1. Open **MobaXterm**.
2. Connect to the Void VM (e.g. `192.168.0.220`, user `root`).
3. In the left-side SFTP browser:
   - Navigate to `/root`.
   - Drag-and-drop `namespace.yaml`, `deployment.yaml`, `service.yaml` from Windows into `/root` (or `/root/_out`, wherever you decide)
4. Confirm they're on the VM: `cd /root`

   `ls`



---

6. Apply resources to the cluster

From the same MobaXterm session (Void VM):

Set KUBECONFIG again (every new session):

```
cd /root/_out

export KUBECONFIG=/root/_out/kubeconfig
```

1. Apply each YAML:

```
./kubectl apply -f /root/namespace.yaml

./kubectl apply -f /root/deployment.yaml

./kubectl apply -f /root/service.yaml
```

2. Check that the pod and service exist:

```
./kubectl -n async-logger get pods

./kubectl -n async-logger get svc
```

3. Expected: If the pod is **`Running`**, deployment is ok.

```
# export KUBECONFIG=/root/_out/kubeconfig
# ./kubectl get nodes
NAME            STATUS   ROLES           AGE     VERSION
talos-pmz-uib   Ready    control-plane   8d      v1.34.1
talos-qrt-zlx   Ready    <none>          8d      v1.34.1
talos-u4y-clt   Ready    <none>          4d20h   v1.34.1
# ./kubectl -n async-logger get pods
NAME                              READY   STATUS    RESTARTS      AGE
async-logger-7c44865b57-s4fnz     1/1     Running   3 (16m ago)   6d19h
# ./kubectl get pods -o wide -n async-logger
NAME                              READY   STATUS    RESTARTS      AGE     IP            NODE            NOMINATED NODE   READINESS GATES
async-logger-7c44865b57-s4fnz     1/1     Running   3 (17m ago)   6d19h   10.244.1.33   talos-qrt-zlx   <none>           <none>
#
```

---

7. Make the service reachable from your laptop (port-forward)

Talos doesn't automatically expose that service outside the cluster, so we use port-forward.

In the **same MobaXterm terminal**:

```
cd /root/_out

export KUBECONFIG=/root/_out/kubeconfig
```

```
./kubectl -n async-logger port-forward svc/async-logger-svc
8080:8080 --address 0.0.0.0
```

You should see:

Important:Keep this terminal open. If you close it, the forwarding stops and your PowerShell calls will fail.

---

8. Test from Windows

Now, from **PowerShell on your Windows host**:

```
Invoke-WebRequest `
  -Method POST `
  -Uri http://192.168.0.220:8080/log `
  -Headers @{ "Content-Type" = "application/json" } `
  -Body '{ "message": "hello from k8s" }'
```

- Replace `192.168.0.220` with your own **Void VM IP**.
- Expected response:
  `{ "message": "hello from k8s!", "status": "queued" }`

If you look back at the **MobaXterm `kubectl port-forward` window**, you'll see "Handling connection for 8080".

To see the worker logs (another MobaXterm tab):

```
cd /root/_out
export KUBECONFIG=/root/_out/kubeconfig
./kubectl -n async-logger logs -l app=async-logger
```

You should see lines like:

- `[WORKER] Starting background worker, waiting for messages...`
- `[WORKER] Processing: hello from k8s!`

That proves:

- HTTP request reached Kubernetes.
- Service routed to the pod.
- Your async worker processed the message.

---

9. Verify in Kubernetes Dashboard

1. In MobaXterm, check the control-plane IP from the Talos UI or `ip a` /
2. In Chrome/Edge on Windows, open: `https://192.168.0.220:8443` `(replace with your own ip address)`
3. Ignore the certificate warning (because it's self-signed) by clicking **Advanced** → **Continue**.
4. When asked for a token:
    - In MobaXterm, go to `/root/_out/dashboard_token.txt` .
    - `cat dashboard_token.txt` and copy the whole token. You can paste into an empty notepad for easy access.
    - Paste into the browser token field.
5. At the top left, select **Namespace: async-logger**.
6. Under **Workloads** → **Deployments / Pods**, you should see:
    - Deployment `async-logger`
    - Pod `async-logger-…` with status `Running`
7. Under **Nodes**, you see both control and worker nodes healthy.

This confirms that everything is running as expected.

---

10. Troubleshooting

10.1 If port-forward fails:

- Ensure the pod is running: `./kubectl get pods`

10.2 If docker push fails:

- Run: `docker login`

10.3 If Talos node is unreachable: Restart VM & ensure bridged network is on the same subnet.